

# bilateral.py — Code Breakdown

Core loop and the features that aid smoother operation

2026-08-25

## Overview

scripts/bilateral.py couples two Damiao DM-series motors on one CAN bus with a virtual spring/damper so they hold a fixed positional relationship. Backdrive either motor by hand and the other follows; hold one still and the reaction force builds at the other.

This document separates the **core mechanism** — the minimum that makes the coupling work — from the **layered features** that keep it smooth, bounded and safe. Line references point into scripts/bilateral.py.

---

## Part 1 — The core loop

### *The mechanism in four lines*

Strip everything else away and the coupling is this (`_tick`, line 157):

```
pa, va = a.pos, a.vel_f          # snapshot both motors
pb, vb = b.pos, b.vel_f
a.step(s*pb + offset, s*vb, ...) # A's setpoint IS B's state
b.step(s*(pa - offset), s*va, ...) # B's setpoint IS A's state
```

### *The host is a relay, not a controller*

The key architectural point: **the host never computes a control law.**

`_Side.step` (line 86) packs a MIT frame and the motor's own firmware evaluates

$$\tau = k_p(p_{des} - p) + k_d(v_{des} - v)$$

against its own encoder, at its own internal rate. The host only reads each motor's position and velocity and cross-assigns them as the other motor's target. That is why 200 Hz is sufficient — the real servo loop is running far faster on-board. POS\_VEL mode cannot do this: its firmware position loop is stiff and has no compliance.

### *One tick, end to end*

```
+ - snapshot a.pos/a.vel_f, b.pos/b.vel_f          (_tick, line 157)
```

```

|
+- a.step(p_des = s*pb + offset, v_des = s*vb)
|   +- clamp p_des to a.pos +/- max_err      (line 94)
|   +- send_command -> MIT frame -> firmware closes the loop
|   +- reply updates a.pos / a.vel / a.vel_f / a.last_heard
|
+- b.step(p_des = s*(pa - offset), v_des = s*va)
|   ^ same snapshot -- NOT a's newly updated position
|
+- _check_alive      0.5 s silence -> abort      (line 163)
+- _status           every 100 ms                (line 171)
+- sleep to next deadline                                     (line 350)

```

`_schedule` (line 129) sits outside the control law. It supplies only the two time-varying quantities the loop needs — `kp_scale` and `offset` — which keeps the ramp logic out of `_tick`.

## The invariant

The loop holds:

$$p_1 = s \cdot p_2 + \text{offset} \quad (s = -1 \text{ with } \text{--mirror})$$

Note what is **not** constrained: the pair's common-mode position. Nothing anchors where the pair sits in absolute terms — only their separation.

---

## Part 2 — Features aiding smoother operation

### A. Streaming rate — keeping frames flowing

| Feature                                | Line   | Failure it prevents                                                                                                                                                                                                                                       |
|----------------------------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5 ms <code>_STREAM_RECV_TIMEOUT</code> | 48, 97 | <code>send_command</code> 's 50 ms default, with two sends per tick, would stall the loop to ~10 Hz. The motor then holds a stale setpoint while the reference moves on, and the next frame lands as a step change — a torque kick that reads as buzzing. |
| Deadline scheduling                    | 350    | A fixed <code>sleep(period)</code> lets tick time drift with send latency. <code>next_tick += period</code> holds the rate; the <code>else</code> branch resyncs rather than accumulating debt when the loop falls behind.                                |
| Tolerating dropped replies             | 98     | <code>if fb is not None</code> — a missed reply keeps the last known state instead of blocking or zeroing it.                                                                                                                                             |

At `--rate 200` the period is 5 ms, but two sends can each block for 5 ms, so under heavy packet loss the loop degrades naturally to about 100 Hz. The `Hz` field in the status line is the read-out for this.

## B. Signal conditioning

The velocity low-pass at line 103 is the non-obvious one:

```
self.vel_f += self.alpha * (fb.vel - self.vel_f)
```

Feedback velocity is 12-bit over  $\pm V_{MAX}$ . On the DM4310 at  $V_{MAX} 50$  that is  $100/4095 \approx$  **0.024 rad/s per count**. That value feeds straight into the partner's  $k_d$  term, so at  $k_d = 2.0$  a single LSB of jitter is roughly 0.05 Nm of torque ripple. The filter smooths it before it becomes force.

--vel-filter trades smoothness against lag. --no-vel-couple is the escape hatch if the pair still buzzes.

## C. Coupling-law correctness

**Snapshot before either send** (line 157). Sending A first and then deriving B's target from A's *updated* position means B sees an error that A has already begun correcting — so A does more of the work every tick, and the pair creeps along the one direction nothing constrains. Simulation showed the correction split going from 2:1 to exactly even once snapshotted.

**Velocity coupling** —  $v_{des}$  is the partner's velocity, not zero. This makes  $k_d$  damp *relative* velocity. With  $v_{des} = 0$  you get absolute damping, which fights all motion including motion you want, and feels viscous and dead. Relative damping only fights divergence, which is what makes the pair feel mechanically connected.

## D. Transient management — four ramps

**Equilibrium captured at enable** (line 323) —  $offset0 = a.pos - s*b.pos$  means zero initial error, so engaging moves nothing. This is the single largest no-jolt feature; everything else is secondary to simply not starting with an error.

**Engage ramp** (line 137) —  $k_p$  goes  $0 \rightarrow$  full, but  $k_d$  is **at full value from the first frame**. Engaging a spring undamped is the dangerous direction.

**Convergence ramp** (line 138) — an explicit --offset only begins moving once the spring is fully up, so the two ramps do not fight each other.

**Release ramp** (line 184) —  $k_p \rightarrow 0$  *while still streaming*. Cutting torque instantly on a stretched coupling would drop a loaded joint; decaying it leaves both motors damped and limp.

## E. Force bounding

The saturating spring at line 94 clamps  $p_{des}$  to within  $max\_err$  of that motor's **own** measured position:

```
p_des = _clamp(p_des, self.pos - max_err, self.pos + max_err)
```

Because the firmware computes  $k_p * (p_{des} - p)$ , clamping the setpoint caps the torque at  $k_p * max\_err$  regardless of how far the partner is dragged. A linear spring would keep building force;

this one plateaus, so yanking one motor cannot command a violent snap-back.

`_gains_for` (line 112) is the asymmetry: the leader keeps its `kp` but takes the softer `bilateral.leader.kd`. Simulation puts this at roughly 55 % less drag under dynamic hand motion, with force reflection unchanged — since reflection is entirely the `kp` term.

## ***F. Failure handling***

`_check_alive` (line 163) aborts after 0.5 s of silence rather than driving a growing spring error into a motor that is no longer answering.

Because both motors receive a frame every tick, the MIT comms-loss watchdog is fed inherently — no idle-hold thread of the kind `scripts/sequence.py` requires.

One ordering subtlety: `except StaleFeedback` must precede `except RuntimeError` (line 361), since it is a subclass.

---

## **Summary**

### **Layer**

`_tick`

`_Side.step`

`_schedule`

`_gains_for`

`_check_alive`

`_release`

main loop

### **Purpose**

The coupling law: cross-assign state between the two motors

Frame packing, setpoint clamping, feedback folding

Time-varying gain scale and equilibrium offset

Role and override resolution for gains

Liveness guard

Graceful torque decay on exit

Timing, telemetry, lifecycle

The design principle throughout: the motor firmware runs the fast loop, the host relays state and shapes the transients.